DRY RUN EVALUATION

Question 1 — Greedy Best-First Search

(a) Step-by-step trace

Greedy Best-First Search expands the OPEN node with the lowest h(n), ignoring the cost already travelled.

Step	Expanded node	OPEN list after expansion (node: h)
1	S	B: 4, A: 8
2	B	E: 2, D: 3, A: 8
3	E	G: 0, D: 3, A: 8
4	G	Goal reached; stop.

(b) Path and cost

Path: S → B → E → G. Total cost = 3 + 6 + 2 = 11.

(c) Is it optimal?

No. The alternative path S → B → D → G costs 3 + 2 + 3 = 8, which is shorter than 11. Greedy search uses only h(n), so it does not guarantee the least-cost path.

Question 2 — A* Search

(a) Step-by-step trace

A* uses f(n) = g(n) + h(n), where g(n) is the path cost from S. OPEN is ordered by increasing f.

Step	Expanded node	OPEN list after expansion (node: g + h = f)
1	S	B: 3 + 4 = 7; A: 2 + 8 = 10
2	B	D: 5 + 3 = 8; A: 2 + 8 = 10; E: 9 + 2 = 11
3	D	G: 8 + 0 = 8; A: 2 + 8 = 10; E: 9 + 2 = 11
4	G	Goal reached; stop.

(b) Path and comparison

A* finds S → B → D → G, cost 3 + 2 + 3 = 8. Greedy Best-First Search found a cost of 11; A* finds the shorter path by considering both g(n) and h(n).

(c) Heuristic admissibility

Node	h(n)	True shortest remaining cost	Check
S	7	8	$7 \leq 8$
A	8	10	$8 \leq 10$
B	4	5	$4 \leq 5$
C	5	6	$5 \leq 6$
D	3	3	$3 = 3$
E	2	2	$2 = 2$

The heuristic is admissible because it never overestimates the true remaining cost. This helps A* find an optimal path when implemented correctly; graph-search A* also needs consistency or node reopening when necessary.

Question 3 — Map Coloring as a CSP

(a) Variables, domains, constraints and graph

Variables: WA, NT, SA, Q, NSW, V. Domain of every variable: {R, G, B}.

Constraints: WA ≠ NT, WA ≠ SA, NT ≠ SA, NT ≠ Q, SA ≠ Q, SA ≠ NSW, SA ≠ V, Q ≠ NSW, NSW ≠ V.

Constraint graph vertices: {WA, NT, SA, Q, NSW, V}. Edges: {(WA,NT), (WA,SA), (NT,SA), (NT,Q), (SA,Q), (SA,NSW), (SA,V), (Q,NSW), (NSW,V)}.

Adjacency list: WA—NT, SA; NT—WA, SA, Q; SA—WA, NT, Q, NSW, V; Q—NT, SA, NSW; NSW—SA, Q, V; V—SA, NSW.

(b) Backtracking with forward checking and MRV

Choose the smallest domain first (MRV); break ties by most unassigned neighbours, then WA, NT, SA, Q, NSW, V. Try colours R, G, B.

Step	Assignment / reason	Domains after forward checking
Initial	All have 3 values; choose SA (degree 5).	WA {R,G,B}; NT {R,G,B}; Q {R,G,B}; NSW {R,G,B}; V {R,G,B}
1	SA = R	WA {G,B}; NT {G,B}; Q {G,B}; NSW {G,B}; V {G,B}
2	Choose NT by degree tie-break; NT = G	WA {B}; Q {B}; NSW {G,B}; V {G,B}
3	MRV singleton tie: Q has more unassigned neighbours than WA; Q = B	WA {B}; NSW {G}; V {G,B}
4	MRV singleton tie: NSW has an unassigned neighbour V; NSW = G	WA {B}; V {B}
5	WA = B	V {B}
6	V = B	All assigned; constraints satisfied.

Final colouring: WA = B, NT = G, SA = R, Q = B, NSW = G, V = B.

(c) Heuristics

MRV chooses the most constrained region first, exposing dead ends early. The degree heuristic prioritises a region that constrains many remaining regions. Forward checking removes incompatible colours immediately, reducing unnecessary trials.

Question 4 — Minimax with Heuristic Evaluation

(a) Backed-up node values

Node	Calculation	Value
Left MIN: first MAX	max(3, 5)	5
Left MIN: second MAX	max(6, 9)	9
Left MIN	min(5, 9)	5
Right MIN: first MAX	max(1, 2)	2
Right MIN: second MAX	max(0, −1)	0
Right MIN	min(2, 0)	0
Root MAX	max(5, 0)	5

(b) Best first move and principal variation

MAX chooses the left branch, guaranteeing 5. The left MIN node chooses its first child (5 rather than 9), then MAX chooses the leaf 5 from (3, 5). Principal variation: Root → left MIN → first MAX child → leaf 5.

(c) Tic-tac-toe evaluation

Assuming the board is X O _ / X _ _ / O _ _, an open line for X contains no O and an open line for O contains no X.

X-open lines: second row, third column, main diagonal = 3. O-open lines: third row, second column, third column, anti-diagonal = 4. Therefore E = 3 − 4 = −1.

Question 5 — Alpha-Beta Pruning

(a) Left-to-right trace

Levels are MAX → MIN → MAX → MIN → leaves. α is MAX's best value so far; β is MIN's best value so far.

Node / leaves	Values examined	α, β information	Result / pruning
Bottom MIN (8,6)	8, 6	β: $+\infty \rightarrow 8 \rightarrow 6$; $\alpha = -\infty$	Returns 6
Bottom MIN (5,7)	5; skip 7	$\alpha = 6$; β becomes 5	$\beta \leq \alpha$; prune leaf 7
Left MAX	6, 5	α becomes 6	Returns 6
Bottom MIN (2,9)	2, 9	Parent $\beta = 6$; returns 2	No cutoff
Bottom MIN (4,3)	4, 3	$\alpha = 2$; $\beta: 6 \rightarrow 4 \rightarrow 3$	Returns 3
Left MIN	Child MAX values 6, 3	β becomes 3	Returns 3; root $\alpha = 3$
Right bottom MIN (6,1)	6, 1	$\alpha = 3$; $\beta: \infty \rightarrow 6 \rightarrow 1$	Returns 1
Right bottom MIN (5,2)	5, 2	$\alpha = 3$; $\beta: \infty \rightarrow 5 \rightarrow 2$	Returns 2; right MAX returns 2
Right MIN	First child value 2	$\alpha = 3$; $\beta = 2$	$\beta \leq \alpha$; prune remaining subtree (9,7,3,4)
Root MAX	Left = 3; right ≤ 2	$\alpha = 3$	Root value = 3

(b) Number of leaves and comparison

Alpha-beta evaluates 11 of 16 leaves and prunes 5: leaf 7 in the left subtree, plus leaves 9, 7, 3, 4 in the remaining right subtree. Plain minimax evaluates all 16 leaves. Both return the same root value, 3.

(c) Move ordering

Good move ordering finds strong MAX values or low MIN values early, causing $\alpha \geq \beta$ cutoffs sooner and increasing pruning. Poor ordering delays cutoffs and evaluates more leaves. Move order affects efficiency, not the final minimax value.